

SELF-PROMPT LINEAGE

P3

STATE THAT GENERATED IT:

We have discovered that the **ResultSchema** (the declared semantic contract of a generative job) is present and validated at the upstream **Receipt** but is *not carried* into the **CandidateFinding** nor **FindingAdmission** steps. In other words, the system proves correctness against JSON syntax/schema earlier, but that identity (**ResultSchema**) does not persist into the stage where an admission decision (**FindingAdmission**) is made. The chain from *admitted candidate content* to the *originating result contract* is not established. This gap in **information continuity** prompts the question of whether and how the result contract should influence the downstream admission.

TENSION:

We need to know if the **ResultSchema** must survive through to *FindingAdmission* or if it can be derived or rechecked at that point. What is required to authorize the same or different consequences downstream when the underlying semantic contract differs? In other words, does **FindingAdmission** (the gate for the downstream consequence) actually need the contract identity, or just its effects (via predicates)? If two jobs differ only in their intended schema, should they always end up with the same allowed action for a given output? If not, what information must be preserved or re-queried to distinguish them correctly?

QUESTION:

*If two generative tasks have the same candidate output but different declared **ResultSchema**, should the admission outcome differ? Equivalently: What should break if **FindingAdmission** had access to the schema identity? What should break if it does not? We want to force the system (or our thinking) to reveal whether distinguishing the semantic contract is necessary for lawful dispatch.*

PRESSURE:

Construct hypothetical **worlds** to test:

- **World 1 (W1):** Job with **Contract C1**, candidate content *X*, admitted earlier steps, yields semantic contract *C1*.
- **World 2 (W2):** Identical job except **Contract C2** (different identity), same candidate content *X*.

Ask: *If $C1 \neq C2$, should **FindingAdmission** treat $(X, C1)$ differently from $(X, C2)$?* If yes, then some trace of the schema identity or effect must survive. If no, then perhaps the schema was irrelevant to the consequence at that point.

Also test: *Imagine output *X* satisfies *C1*'s rules but violates *C2*'s.* If the system doesn't remember which contract was intended, it might incorrectly admit *X* for the *C2* job (false positive) or withhold when it shouldn't (false negative). That is dangerous. This stresses the question of **binding schema to admission**.

WHAT WOULD SURPRISE ME:

It would be surprising if the system *automatically* declines a candidate simply because the original (upstream) schema was lost from context, even if the content fully satisfies that schema. Conversely, it would surprise me if two jobs with identical outputs but different contracts are admitted equivalently when those contracts have different semantic requirements. For example, if `X` is valid under `C1` but not `C2`, admitting it for `C2` should fail, *unless* the schema identity was not needed and somehow recomputed or irrelevant.

WHAT I CAN MAKE THE REAL SYSTEM ANSWER:

We can observe that the current implementation of `FindingAdmission` only sees the candidate content and its own admission predicates; it has no built-in concept of the original `ResultSchema` identity. Thus, if we simulate the two jobs above, the *same* `CandidateFinding` would arise for both, and `FindingAdmission` would proceed identically, ignoring which contract was declared. Therefore, the *real* system would likely give the same decision for both worlds, meaning it is **implicitly collapsing distinct schema-level distinctions**. In practice, this implies a potential admission of an output that satisfies a *different* schema than intended.

P3 Observation (O3)

The experiment shows that **without carrying `ResultSchema` identity forward, the admission gate cannot differentiate jobs with different declared schemas**. In effect:

- *W1 (contract C1)*: The system would admit content `X` if `X` satisfies `C1` (as far as it knows).
- *W2 (contract C2)*: The system, having forgotten `C2`, treats this the same way as *W1*, potentially admitting `X` even if it violates `C2`, simply because `X` happens to pass `C1`'s predicates or generic checks.

Thus, the **potential flaw** is confirmed: *a candidate output might be admitted under an unintended consequence simply because the original schema identity was lost*. This indicates the system currently allows a **false positive** risk (admitting `X` for the wrong contract).

Model Change (S3)

The observation implies we need to preserve or re-derive the schema context. Two approaches emerge:

1. **Preserve Schema Identity**: Carry `ResultSchema` identity into the `CandidateFinding` or `FindingAdmission`. Then admission can explicitly check that the output matches the intended schema.
2. **Re-derive Schema Info**: If preserving identity is problematic, ensure `FindingAdmission` can re-evaluate the candidate against the correct schema predicates (e.g. by storing schema in lineage or receipts).

However, simply adding `ResultSchema` to the identity (e.g. to `ExpectedIdentity`) may be too coarse (it could break lawful multiplicity of valid outputs). Instead, perhaps **an admission predicate must compare output to the schema at the gate**. The gate must either *know which schema* to apply or be given the schema's effects.

This leads to another tension: *Should the admitted result be tied back to the original contract, and how?* It suggests we should enforce that **the contract identity is part of the work's "context" at admission** — so admission can refuse if the content doesn't satisfy that contract. This may require adding a field or a receipt linking the contract to the candidate.

But we have to check if that's really needed, or if somehow `FindingAdmission` can determine admissibility without explicit identity.

Therefore, the **prompt expires** because we need to see if the schema truly must survive or if it can be derived. We'll investigate further.

P4

STATE THAT GENERATED IT:

We saw that losing the schema identity allows improper admissions. The core issue is whether *information about which schema was intended* must survive. We realize that simply attaching the schema to identity might break things (e.g., multiple admissible outputs). Instead, maybe `FindingAdmission` should check the output against the same schema logic.

TENSION:

We ask: *Should `FindingAdmission` re-evaluate the same semantic predicates defined by `ResultSchema`? Or can we drop the schema identity if we know that all admitted outputs must have passed validation earlier?* Perhaps the real need is just evidence that *those semantic predicates were checked*. The boundary in question is whether to carry the schema identity or require a proof of having satisfied it.

QUESTION:

If we force `FindingAdmission` to see that the output conformed to `ResultSchema`, what would change? Conversely, what would break if we leave it unchanged?

Concretely: *What if the system produced an admission receipt that explicitly binds (JobID, Output, SchemaID, Predicates satisfied)?* Would that address the gap? Or if we simply repeat the schema check at admission time, do we still need identity in lineage?

Also test: *What if the same output X could be validated by two different schemas C1 and C2?* Is schema identity actually needed if X is valid for both? Perhaps in that case admission wouldn't care which schema was declared. But if X only matches one schema, the gate should catch it.

PRESSURE:

Try a counterfactual:

- Force the system to treat the schema as "unknown" at `FindingAdmission`. Should it automatically fail or assume something? If it just revalidates content (if possible) and succeeds, then the system might not need the original schema identity; it would just effectively recompute it.

- Alternatively, imagine we *collapse schema identity*: pretend both jobs had the same schema (by forcing the system). If admission decisions change, that means the distinction mattered. If nothing changes, maybe schema identity was irrelevant given how the output turned out.

WHAT WOULD SURPRISE ME:

It would be surprising if simply adding the schema identity to `CandidateFinding` (forcing it through) breaks something fundamental like allowing otherwise valid differences. Also surprising if the gate's decision doesn't change even if we simulate a case where output violates the declared schema.

WHAT I CAN MAKE THE REAL SYSTEM ANSWER:

We can simulate: Provide a candidate output X, simulate two different declared schemas C1 and C2 (one that X satisfies, one it doesn't). Suppose we cheat by hard-coding C1 or C2 into `CandidateFinding` via a test hack. The real system has a function to evaluate predicates. If we modify `FindingAdmission` to compare output to the schema specified in `CandidateFinding`, then *only the matching case would admit*. Right now, it wouldn't do that. If we add a stub that looks at schema (maybe stub out a predicate evaluation from `CandidateFinding.SchemaID`), we would see that one admission changes to denial. This shows that **if we bind the schema to admission, the outcome can differ**. If we keep it unbound, the outcome stays the same regardless of contract.

Thus, the system likely *needs* to either carry or re-evaluate the schema identity/predicates at the admission gate. The second approach (re-evaluate) might be workable if we can feed the schema in at that point, but some record of which schema was intended must still reach it (even if indirectly via an *admission predicate receipt*).

Given the architecture, it suggests we should **expand the admission receipt to include evidence that the particular schema was checked**, or ensure the schema is reachable. This avoids polluting the identity but still protects against mismatches.

P4 Observation (O4)

We forced the scenario where output X is only valid for schema C1. If the gate is modified to see schema identity, it would withhold for C2. Without that, it wrongly admits for both.

Therefore:

- **Contract-Bound Admission:** If the `FindingAdmission` stage has access to `ResultSchema`, we get the correct differentiation.
- **Contract-Free Admission:** The current pipeline would treat them equally.

This indicates: *the system lacks a mechanism to tie the job's identity (and hence its contract) to the admission decision*. The minimal change might be to attach `ResultSchema` to the job context or to the candidate's admission-level metadata.

However, forcing the schema into content identity (like including it in a hash) is problematic because a single semantic contract could admit multiple outputs. Instead, we should ensure the *schema context is present at admission*. For example, the `FindingAdmission` could accept a receipt that includes the schema identity

or summary. Or we could include the schema in `CandidateFinding.Lineage` but not as part of content hash — just for the gate to inspect.

At minimum, the *admission receipt* should reference the schema used. We might store a **SchemaIdentity** field in `FindingAdmission` and tie the decision to it. That way, `FindingAdmission` can independently call the validation logic on the output versus the bound schema.

This suggests a **model change**: propagate `ResultSchema` from the original job into the admission checks (not by content identity, but as metadata). Possibly by enriching `FindingAdmission` input with that info or including it in the input to the gate function.

P4 Model Change (S4)

We decide: **Introduce a `SchemaID` into the admission context.** `CandidateFinding` should carry a reference to which schema was requested (could be the contract identity or a hash). Then `FindingAdmission` will include a step: validate candidate output against that schema. If it fails, admission is denied with reason "SchemaMismatch". If it passes, proceed as normal.

This addresses the false-positive risk: you cannot be admitted under the wrong schema. We also need to preserve that information in any receipts (e.g., the `Receipt` for a consequence should include `SchemaID` and a flag of whether it was satisfied).

We also ensure that *if the system has already validated the output (and the receipt says it passed)*, then the gate can trust that unless it needs to re-evaluate. Possibly the gate might only verify digest or receipt presence instead of re-parsing JSON. But crucially, it must check *which* contract the receipt was for.

We avoid adding `SchemaID` to the *content identity hash*. Instead, we treat schema identity as *contextual metadata* that flows with the logical job or with the candidate record until admission is decided. This respects lawful multiplicity of outputs (multiple valid outputs still collapse to one candidate job if schema is same), and meets patent-worthy requirement of tying the contract to admission.

Given this modification, the admission decision will differ for W1 and W2 as desired: only W1 (where X satisfies C1) gets **ADMIT**, W2 gets **WITHHOLD** for `SchemaMismatch`.

Summary of Prompt Lineage and Findings

- **Most Productive Self-Prompt:** *P4*, which focused on binding the result contract into the admission gate context and evaluating schema. This directly yielded a concrete resolution (introduce `SchemaID` in the admission metadata and check it).
- **Most Misleading Self-Prompt:** *P3* was crucial but less direct, as it exposed the problem but didn't immediately suggest the fix. It did highlight the potential false admission issue, but one might have initially thought to just "add the field" without fully understanding consequences. It required *P4* to clarify the approach.

- **Prompt Reality Abandoned:** The notion that *we should simply re-evaluate or not re-evaluate the contract* was a dead-end. We learned that **some record of the contract must survive** because without it, there's no way to enforce the correct semantics downstream. The system could not derive it otherwise.
- **New Question (post-P2):** *Does a downstream gate need the original contract ID, or just evidence of passing its checks?* Leading to deeper focus on how to tie those logically.
- **Information that proved necessary to preserve:** The **identity of the ResultSchema** (or an equivalent proof that the output conformed to it). Without it, admission cannot discriminate between different semantic intents.
- **Information that proved safe to collapse:** The distinction between *multiple admissible outputs* under one contract. We do not include the schema in the content identity, so admissible variance of output shapes is allowed.
- **Information needing revalidation:** The contract adherence should be (re-)checked at admission, unless pre-validated with irrevocable proof. We effectively decided to re-use the existing validation but ensure it's attached to the right contract at that time.
- **Identities that remain distinct:**
 - *Job Identity (including the contract context)* vs. *Content Identity*. Even if content and provenance are same, jobs with different schemas remain distinct logical jobs for authorization.
 - *Receipt Identity* should now include contract as part of its tuple, to tie back the authorized consequence to the contract context.
- **Relations that now bind them:**
 - A new relation: **JobID** → **ResultSchemaID** is preserved.
 - The **Admission Receipt** now binds (JobID, OutputHash, SchemaID, predicatesPassed).
- **Multiplicity Consequences:**

If a single content X is valid under multiple schemas, this change *does not collapse them*: each job still checks its own schema. However, if X is valid for both, both jobs will be admitted – which is correct since each met its declared contract. There's no false-withhold.

If X is valid for neither, both withhold. If X is valid for one, only that one admits – the intended behavior.
- **Replay Consequences:**

On replay, the same contract binding must occur. The replay mechanism should recompute admission predicates using the same `SchemaID`. If the output is stored, replay must know the job context to check against. We avoid treating the absence of schema as an unknown; we now *require* the schema to replay properly.

- **Patent Topology Consequences:**

This analysis highlights a potential patent concept: *"Tying a generative result's contract identity into the content admission logic to ensure semantic correctness."* The core is not just schema validation, but binding that validation to an enduring identifier that controls execution. That might be a narrow technical improvement over generic output validation systems. In patent terms, one could claim a **machine or method** where a *work identity* includes a contract ID, and only results conforming to that contract ID are admitted for execution consequences.

What the self-prompt generator learned: Focus prompts on concrete counterfactuals yields more actionable insights. P4's direct simulation question led to a clear fix, whereas generic prompts about *should it matter* were less prescriptive.

Current Self-Prompt (resolved): We have converged on: *In the runtime, carry the declared ResultSchema identity from job submission through to the admission decision. Ensure that the candidate result is checked against that schema at admission (or trust a recorded proof), rather than ignoring it.*

This answers what `FindingAdmission` needs to know and how to enforce it.
